

Differential Geometry MATH3D

2026-01-24, 16:50

1 Geodesic Path on Surface (Mesh vs. Implicit)

1.1 Problem: “edge-walk” zig-zag artifacts

The current path looks like a jagged polyline because it is effectively a *shortest path constrained to mesh edges* (i.e., Dijkstra/A* on the vertex-edge graph). On coarse triangle meshes (e.g., Marching Cubes output), this produces visible “staircasing” and corner kinks.

1.2 Practical solver options (easy win → gold standard)

A* / Dijkstra on a richer graph (cheap upgrade, still graph-based)

Goal: reduce zig-zag by allowing paths to cut across faces, not only follow edges.

Add face diagonals / local shortcuts. Add extra edges such as:

- connect vertices across quads (if you can detect them),
- connect 2-ring neighbors (neighbors-of-neighbors) with weight equal to straight Euclidean distance.

Pros: simple, fast.

Cons: still not a true geodesic; can still kink.

Build a “visibility graph” on the 1-ring. For each vertex, connect it to all vertices that appear in adjacent faces (not only direct mesh edges).

- This effectively lets A* shortcut within local face neighborhoods.

Pros: fewer zig-zags.

Cons: can “cut corners” incorrectly on coarse meshes.

This is the easiest solver change if minimal math is required.

Exact geodesics on triangle meshes (classic discrete geodesics)

These compute *exact shortest paths* on polyhedral (triangulated) surfaces.

Chen–Han algorithm. Exact shortest paths on polyhedral surfaces.

- **Pros:** exact on the mesh surface.
- **Cons:** complex to implement robustly.

Mitchell–Mount–Papadimitriou (MMP). Also exact on polyhedral surfaces.

- **Pros:** exact.
- **Cons:** even more complex than Chen–Han.

Heat Method (best practical solver for apps)

This is the *go-to* geodesic distance solver in graphics: smooth, robust, and relatively straightforward if sparse linear solves are available.

High-level pipeline. Let L be the Laplace–Beltrami operator (discrete cotan Laplacian) and M the mass matrix.

1. **Heat diffusion (short time t):**

$$(M + tL)u = M\delta_s$$

where δ_s is a point source at the start vertex (or barycentric point source).

2. **Normalize gradient field:**

$$X = -\frac{\nabla u}{\|\nabla u\|}$$

3. **Poisson solve to obtain distance ϕ :**

$$L\phi = \nabla \cdot X$$

4. **Extract the geodesic curve:** backtrace from target to source along $-\nabla\phi$ (integrate on the surface).

Pros: excellent results, handles noisy/coarse meshes well, widely used.

Cons: requires L/M construction + sparse linear system solves.

Fast Marching Method (FMM) on the mesh

Compute distance by propagating a front on the triangulated surface.

- **Pros:** more accurate than edge Dijkstra; classic PDE approach.
- **Cons:** harder than A*; often less pleasant than Heat Method in practice.

Intrinsic triangulation / edge flips (high quality, more work)

Build an *intrinsic Delaunay triangulation* (IDT) to improve geodesic stability on poor meshes, then run geodesic algorithms on the intrinsic mesh.

- **Pros:** accurate and stable even on low-quality triangulations.
- **Cons:** significant implementation effort.

1.3 What to do for Marching Cubes implicit surfaces

Immediate fix (short-term)

Keep the welded mesh graph, but improve the path quality:

- add face-based start/end nodes (barycentric placement) if not already,
- add 2-ring edges to reduce zig-zag,
- resample the path to uniform arc-length steps,
- Laplacian-smooth the polyline (few iterations),
- project smoothed points back onto the implicit surface $f(x, y, z) = 0$ (Newton projection),
- do this *uniformly across all implicit surfaces* (not only spheres).

Real solver (next milestone)

Implement the **Heat Method** on the welded indexed mesh:

- produces a smooth distance field ϕ ,
- extracting the path via backtracing yields a clean, stable geodesic,
- directly addresses the zig-zag “edge-walk” artifact.

1.4 Geodesics directly on an implicit surface $f(x, y, z) = 0$ (ODE formulation)

Let

$$S = \{x \in \mathbb{R}^3 : f(x) = 0\}, \quad g(x) = \nabla f(x), \quad n(x) = \frac{g(x)}{\|g(x)\|}.$$

A curve $x(s) \in S$ is a geodesic (parameterized by arc-length s) iff:

- it stays on the surface: $f(x(s)) = 0$,
- its acceleration is purely normal: $x''(s) \parallel n(x(s))$,
- and $\|x'(s)\| = 1$ (arc-length parameterization).

Constraint differentiation

Differentiate $f(x(s)) = 0$:

$$\nabla f(x) \cdot x' = 0,$$

and again:

$$x'^T H_f(x) x' + \nabla f(x) \cdot x'' = 0,$$

where $H_f(x)$ is the Hessian matrix of f .

Assume geodesic acceleration is normal:

$$x'' = \lambda(s) n(x) = \lambda(s) \frac{\nabla f(x)}{\|\nabla f(x)\|}.$$

Then

$$\nabla f(x) \cdot x'' = \lambda(s) \|\nabla f(x)\|.$$

Therefore

$$\lambda(s) = -\frac{x'^T H_f(x) x'}{\|\nabla f(x)\|}.$$

Final ODE system (arc-length geodesic)

Let $v = x'$ so that $\|v\| = 1$. The geodesic ODE becomes:

$$\begin{cases} x' = v, \\ v' = -\frac{v^T H_f(x) v}{\|\nabla f(x)\|^2} \nabla f(x), \end{cases}$$

with constraints

$$f(x) = 0, \quad v \cdot \nabla f(x) = 0, \quad \|v\| = 1.$$

Two-point boundary condition and shooting

To connect p_1 to p_2 , choose the initial tangent direction $v(0)$ so that the integrated curve hits p_2 . This is a boundary value problem, typically solved by **shooting**:

1. Build an orthonormal tangent basis at p_1 : $(e_1, e_2) \subset T_{p_1}S$ with

$$e_i \cdot n(p_1) = 0, \quad e_1 \cdot e_2 = 0, \quad \|e_i\| = 1.$$

2. Parameterize direction by an angle:

$$v(0; \theta) = \cos \theta e_1 + \sin \theta e_2.$$

3. Integrate to length L to obtain $x(L; \theta)$.

4. Optimize (θ, L) to match p_2 :

$$E(\theta, L) = \|x(L; \theta) - p_2\|^2.$$

Practical issues.

- Multiple geodesics can exist between two points.
- Near the cut locus, the minimization is ill-conditioned.
- Numerical drift can push (x, v) off the manifold without stabilization.

1.5 Stability: keep the curve on the surface

ODE integration introduces drift; apply projection steps every step (or every few steps).

Newton projection of position onto $f(x) = 0$

$$x \leftarrow x - \frac{f(x)}{\|\nabla f(x)\|^2} \nabla f(x)$$

Project velocity into the tangent plane and renormalize

$$v \leftarrow v - (v \cdot n) n, \quad v \leftarrow \frac{v}{\|v\|}$$

1.6 Analytic gradient ∇f for implicit surfaces

If $f(x, y, z)$ is given as an explicit expression, then

$$\nabla f(x, y, z) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right)$$

Implementation options

1. **Symbolic differentiation:** parse f into an AST and build ASTs for f_x, f_y, f_z .
2. **Automatic differentiation (recommended):** forward-mode AD (dual numbers) with three derivative slots (dx, dy, dz) .
3. **Finite differences (fallback):**

$$f_x(x, y, z) \approx \frac{f(x+h, y, z) - f(x-h, y, z)}{2h}$$

similarly for y, z .

Manual examples

If

$$f(x, y, z) = x^2 + y^2 + z^2 - 1, \quad \text{then} \quad \nabla f = (2x, 2y, 2z),$$

and if

$$f(x, y, z) = x \sin(y) + e^z, \quad \text{then} \quad \nabla f = (\sin(y), x \cos(y), e^z).$$

1.7 Recommendation

- **Shippable solution:** triangulate (Marching Cubes) $\rightarrow$ Heat Method on the mesh $\rightarrow$ backtrace to obtain the path.
- **Experimental “pure implicit” mode:** ODE + shooting + projection each step (more fragile but elegant).

2 Math3D Surface Meshing Strategy (Implicit / Explicit / Parametric / Weierstrass)

2.1 Goal

Keep meshing **robust**, **predictable**, and **UI-consistent** across all surface modes in Math3D. The key design choice is to *avoid voxelization for smooth parametric surfaces* and reserve CGAL/VTK for implicit surfaces where topology is not given by a parameter domain.

2.2 Parametric-family surfaces (Explicit, Parametric, Weierstrass): do *not* use VTK/CGAL by default

For any surface that can be evaluated as a smooth map

$$X(u, v) \in \mathbb{R}^3,$$

do not use VTK/CGAL at all as the default.

Parametric mode

$$X(u, v) \in \mathbb{R}^3.$$

Explicit mode

Explicit graphs are parametric surfaces in disguise:

$$z = f(x, y) \implies X(u, v) = (u, v, f(u, v)).$$

So explicit surfaces reuse the exact same pipeline as parametric surfaces.

Weierstrass mode

The Weierstrass representation outputs a parametric surface over a domain (with integration / stability checks), so the final mesh generation remains parametric:

$$X(u, v).$$

Meshing rule: triangulate the parameter domain and map through $X(u, v)$.

2.3 Robust default: domain triangulation + adaptive refinement

Default pipeline for Parametric / Explicit / Weierstrass:

- 1) Start with a coarse triangulation in parameter space (u, v) (grid triangulation or Delaunay in 2D).
- 2) Map vertices via $X(u, v)$ into $\mathbb{R}^3$.
- 3) Refine triangles adaptively where needed, based on one or more criteria:
 - edge length in $\mathbb{R}^3$ too large,
 - normal variation across the triangle too large,
 - curvature estimate too large / rapidly changing,
 - proximity to known singular regions (optional).
- 4) Compute normals / curvature / derived fields as usual.

Why this is robust.

- Avoids topology surprises introduced by voxelization.
- Gives deterministic control over density exactly where needed (high curvature, seams, singularities).
- Works uniformly for Explicit/Parametric/Weierstrass and matches the current renderer model.

2.4 Implicit surfaces $f(x, y, z) = 0$: CGAL is the quality gold standard

Implicit surfaces do *not* come with a natural parameter domain:

$$S = \{(x, y, z) \in \mathbb{R}^3 : f(x, y, z) = 0\}.$$

Meshing is fundamentally different, and external meshing libraries become the robust choice.

Two implicit families

A) Analytic implicit $f(x, y, z)$ (best quality: CGAL / pygalmesh). If you have an analytic implicit function $f(x, y, z)$, the best mesh quality and robustness usually comes from:

- CGAL implicit surface meshing (via pygalmesh or direct CGAL bindings).

B) Sampled scalar field / voxels / SDF volume (best speed: VTK FlyingEdges / Marching Cubes). If the input is already a discretized volume (grid samples / voxel SDF), the best engineering path is:

- VTK isosurfacing (FlyingEdges3D / Marching Cubes).

Conclusion for implicit meshing.

Quality: CGAL > VTK Speed / preview: VTK > CGAL.

2.5 One-pipeline rule for predictable UX in Math3D

Use a single, easy-to-explain rule:

Use param-domain adaptive triangulation for

- Explicit surfaces ($z = f(x, y)$ as $X(u, v)$),
- Parametric surfaces ($X(u, v)$),
- Weierstrass surfaces ($X(u, v)$ from the integration pipeline).

Use CGAL / pygalmesh for

- Analytic implicit surfaces $f(x, y, z) = 0$ **when mesh quality matters.**

Use **VTK isosurfacing** for

- Volumetric / SDF preview meshes,
- draft implicit meshes when you already have a 3D grid.

2.6 Implications for the “region $\rightarrow$ python $\rightarrow$ return mesh” workflow

Assume the framework can send a selected region and get back a triangle mesh $\{V, F\}$.

Parametric / Explicit / Weierstrass

- Python is not required for baseline meshing.
- Mesh can be generated deterministically in JS from the selected parameter region:

(u, v) -region or (x, y) -region.

- Python may still be useful for heavy adaptive refinement or offline-quality meshing, but it is optional.

Implicit

- Region selection provides:
 - bounding box (AABB) in $\mathbb{R}^3$,
 - target edge length / sizing control,
 - max triangles / budget.
- Python runs CGAL meshing inside the bounding box and returns a mesh:

$\{V, F\}$.

2.7 “Mesh quality slider” mapping (consistent UX)

Map a single *Mesh Quality* control to the correct notion of resolution in each mode:

- **Parametric / Explicit / Weierstrass:** refinement thresholds in parameter-space triangulation (edge length, normal variation, curvature variation, refinement depth).
- **Implicit (CGAL quality mode):** sizing field / target edge length / approximation error.
- **Implicit (VTK preview mode):** voxel/grid resolution (and optionally smoothing iterations).

2.8 Summary

- **Do not voxelize** parametric-family surfaces by default: mesh the domain, map through $X(u, v)$, adaptively refine.
- **Use CGAL** for analytic implicit $f(x, y, z) = 0$ when you want the best surface quality.
- **Use VTK** for volumetric/SDF preview meshes and fast previews/drafts.
- **Keep one UI mental model:** one quality slider that controls refinement in the appropriate space.

Heat Method Geodesics on a Triangulated Patch (Implementation Notes)

We assume a triangle mesh patch

$$V = \{p_i\}_{i=1}^N \subset \mathbb{R}^3, \quad F = \{(i, j, k)\} \subset \{1, \dots, N\}^3,$$

with oriented faces $f = (i, j, k)$. A point x lying in a face f is represented by

$$x = b_i p_i + b_j p_j + b_k p_k, \quad (b_i, b_j, b_k) \in \Delta^2, \quad b_i + b_j + b_k = 1, \quad b_\ell \geq 0,$$

i.e. by its *barycentric* coordinates in that face.

Precompute and cache per mesh

All quantities below are cached *per mesh* (keyed by a mesh hash) so that repeated geodesic queries reuse the same operators and sparse factorizations.

Mean edge length. Let E be the set of (undirected) mesh edges. The mean edge length is

$$\bar{h} = \frac{1}{|E|} \sum_{(a,b) \in E} \|p_a - p_b\|.$$

Face areas and normals. For each face $f = (i, j, k)$, define

$$n_f = (p_j - p_i) \times (p_k - p_i), \quad A_f = \frac{1}{2} \|n_f\|.$$

Barycentric basis gradients. Let $\lambda_i, \lambda_j, \lambda_k$ be the barycentric basis functions on face f . Their gradients are constant on f and can be written as

$$\nabla \lambda_i = \frac{n_f \times (p_k - p_j)}{2A_f}, \quad \nabla \lambda_j = \frac{n_f \times (p_i - p_k)}{2A_f}, \quad \nabla \lambda_k = \frac{n_f \times (p_j - p_i)}{2A_f}.$$

These three vectors are cached per face and reused to compute gradients of any piecewise-linear scalar field.

Cotan Laplacian L . For each undirected edge (a, b) shared by one or two faces, let α, β be the angles opposite (a, b) in those incident faces (only α exists on the boundary). Define the cotan weight

$$w_{ab} = \frac{1}{2}(\cot \alpha + \cot \beta) \quad (\text{use only the existing angle(s) if the edge is on the boundary}).$$

The cotan Laplacian $L \in \mathbb{R}^{N \times N}$ is assembled as

$$L_{ab} = -w_{ab} \quad (a \neq b), \quad L_{aa} = \sum_{b \sim a} w_{ab},$$

where $b \sim a$ ranges over vertices adjacent to a .

Lumped mass matrix M . Let M be diagonal with entries obtained by area lumping: for each face $f = (i, j, k)$,

$$M_{ii} += \frac{A_f}{3}, \quad M_{jj} += \frac{A_f}{3}, \quad M_{kk} += \frac{A_f}{3}.$$

Face adjacency. For each face $f = (i, j, k)$, build the neighbor across each of its three edges. Equivalently, store

$$\text{nbr}[f][0] = \text{face across edge } (j, k), \quad \text{nbr}[f][1] = \text{face across edge } (k, i), \quad \text{nbr}[f][2] = \text{face across edge } (i, j)$$

with value -1 on boundary edges.

Cached sparse factorizations. For fixed t (defined below) cache a sparse factorization of

$$A = M + tL.$$

For the Poisson solve cache a factorization of a *pinned* Laplacian L_p , where one vertex p is fixed to remove the nullspace.

Heat solve

Time step. Choose

$$t = t_{\text{factor}} \bar{h}^2.$$

Barycentric delta. Let the source point lie in face $f_s = (i, j, k)$ with barycentric coordinates (b_i, b_j, b_k) . Form a discrete delta vector $\delta \in \mathbb{R}^N$ by

$$\delta_i += b_i, \quad \delta_j += b_j, \quad \delta_k += b_k,$$

and $\delta_\ell = 0$ for all other vertices ℓ .

Heat equation. Solve for $u \in \mathbb{R}^N$:

$$(M + tL)u = \delta.$$

Vector field

Per-face gradient. For any vertex scalar field $s \in \mathbb{R}^N$, the gradient on face $f = (i, j, k)$ is

$$\nabla s|_f = s_i \nabla \lambda_i + s_j \nabla \lambda_j + s_k \nabla \lambda_k.$$

Compute $\nabla u|_f$ for every face.

Normalize. Define the per-face unit vector field

$$X_f = -\frac{\nabla u|_f}{\|\nabla u|_f\|},$$

with a small ε safeguard (if $\|\nabla u|_f\| < \varepsilon$, set $X_f = 0$).

Poisson solve for distance

Discrete divergence. Given a face-constant vector field X_f , assemble a vertex RHS $b \in \mathbb{R}^N$ by distributing each face contribution via the barycentric gradients: for each face $f = (i, j, k)$,

$$b_i += -A_f X_f \cdot \nabla \lambda_i, \quad b_j += -A_f X_f \cdot \nabla \lambda_j, \quad b_k += -A_f X_f \cdot \nabla \lambda_k.$$

Pinned Laplacian. Since L has a constant-function nullspace on each connected component, pin one vertex index p (e.g. $p = 1$) by enforcing

$$\phi_p = 0.$$

In practice, replace row p by the identity constraint:

$$(L_p)_{p*} = 0, \quad (L_p)_{pp} = 1, \quad b_p = 0.$$

Poisson equation. Solve for $\phi \in \mathbb{R}^N$:

$$L_p \phi = b.$$

Geodesic path backtrace (continuous, face-walk)

Let the target point lie in face f_0 with barycentric coordinates $b^{(0)} = (b_1^{(0)}, b_2^{(0)}, b_3^{(0)})$. We march a point x across faces following the (negative) gradient of ϕ .

Step size. Set

$$h = \text{step_factor} \cdot \bar{h}.$$

Face direction. At the current face $f = (i, j, k)$, compute

$$g_f = \nabla \phi|_f = \phi_i \nabla \lambda_i + \phi_j \nabla \lambda_j + \phi_k \nabla \lambda_k, \quad d = -\frac{g_f}{\|g_f\|}.$$

Barycentric update. Because barycentric coordinates are linear on each face,

$$\Delta b_\ell = \nabla \lambda_\ell \cdot (h d), \quad \ell \in \{i, j, k\}.$$

Thus the tentative barycentric coordinates are

$$b'_\ell = b_\ell + \nabla \lambda_\ell \cdot (h d).$$

Edge crossing. If all $b'_\ell \geq 0$, then the new point remains in the same face. If some component becomes negative, we intersect the first crossed edge. For any ℓ with $b'_\ell < 0$, define the crossing fraction

$$\alpha_\ell = \frac{b_\ell}{b_\ell - b'_\ell},$$

and take $\alpha = \min_\ell \alpha_\ell$ (the first boundary hit). Update to the edge point

$$b \leftarrow b + \alpha (b' - b),$$

so that one barycentric coordinate is 0 (on the crossed edge). Let ℓ^* be the index attaining the minimum; then the crossed edge is the one *opposite* vertex ℓ^* . Move to the adjacent face using the cached adjacency:

$$f \leftarrow \text{nbr}[f][\ell^*].$$

If $\text{nbr}[f][\ell^*] = -1$, a boundary edge was hit and the march stops (v1).

Monotonicity / tie-break toward the source. To ensure the march moves toward the source level set, accept only steps that decrease ϕ (equivalently, move along $-\nabla \phi$). If numerical noise yields ambiguity, break ties by also choosing the direction that decreases Euclidean distance to the source point.

Stopping. Stop when either

$$\|x - x_s\| < \varepsilon_{\text{pos}} \quad \text{or} \quad |\phi(x) - \phi(x_s)| < \varepsilon_\phi \quad \text{or} \quad \text{steps} > \text{max_steps}.$$

The visited points x form the geodesic polyline, and its length is the sum of segment lengths.

Caching summary. For each mesh patch we cache:

$$(L, M, \{A_f\}, \{\nabla \lambda\}, \text{nbr}, \bar{h}, \text{LU}(M + tL), \text{LU}(L_p)),$$

so repeated queries reuse all operators and sparse factorizations.

3 Example 1: Zariski topology on $\mathbb{A}^1$ and the product topology on $\mathbb{A}^1 \times \mathbb{A}^1$

Assume k is algebraically closed.

3.1 Affine line $\mathbb{A}^1$

Let

$$\mathbb{A}^1 = \text{Spec}(k[t]).$$

For an ideal $I \subset k[t]$ define the algebraic set

$$Z(I) = \{a \in k \mid f(a) = 0 \text{ for all } f \in I\}.$$

3.1.1 Closed sets

Every ideal in $k[t]$ is principal:

$$I = (f).$$

Then

$$Z(f) = \{a \in k \mid f(a) = 0\}.$$

Over an algebraically closed field, a nonzero polynomial f has finitely many roots, hence:

closed sets in $\mathbb{A}^1$ are $\emptyset$, $\mathbb{A}^1$, and finite subsets of k .

3.1.2 Basic open sets

For $f \in k[t]$ define

$$D(f) = \mathbb{A}^1 \setminus Z(f).$$

Then:

$D(f)$ is open, and these $D(f)$ form a basis.

Example:

$$f(t) = t(t-1), \quad Z(f) = \{0, 1\}, \quad D(f) = k \setminus \{0, 1\}.$$

3.2 Product topology on $\mathbb{A}^1 \times \mathbb{A}^1$

Give each factor $\mathbb{A}^1$ the Zariski topology (cofinite topology). Let

$$X = \mathbb{A}^1 \times \mathbb{A}^1$$

with the product topology.

3.2.1 Basic opens and basic closed sets

A basis of opens is

$$U \times V$$

where $U, V \subset \mathbb{A}^1$ are Zariski open. Since U, V are complements of finite sets, any basic open has the form

$$(k \setminus F) \times (k \setminus G)$$

with F, G finite.

A typical closed set in the product topology is a finite union of sets of the form

$$F \times \mathbb{A}^1, \quad \mathbb{A}^1 \times G, \quad F \times G$$

with F, G finite.

3.2.2 Comparison with the Zariski topology on $\mathbb{A}^2$

In $\mathbb{A}^2$ (Zariski topology coming from $k[x, y]$), the diagonal

$$\Delta = Z(y - x)$$

is Zariski closed. But Δ is not closed in the product topology, because product-closed sets are built from finitely many vertical and horizontal fibers and finite sets.

Hence:

product topology on $\mathbb{A}^1 \times \mathbb{A}^1$ is strictly coarser than the Zariski topology on $\mathbb{A}^2$.